



JUNTANDO COLEÇÕES E AGREGAÇÃO

BANCO DE DADOS NÃO RELACIONAIS

CST em Desenvolvimento de Software Multiplataforma



PROF. Me. TIAGO A. SILVA



COMO JUNTAR DUAS COLEÇÕES?

Estratégias de Junção de Documentos

COMO JUNTAR DUAS COLEÇÕES?

- No MongoDB, podemos relacionar duas coleções usando **referências** entre documentos ou **incorporando** (embed) documentos dentro de outros.
- A escolha entre essas abordagens depende do caso de uso e da necessidade de normalização dos dados:
 - Referências (references)
 - Documentos Incorporados (embedding)

UMA COLEÇÃO DENTRO DA OUTRA

Guardar dados usando conceito de Embedding

COMO JUNTAR DUAS COLEÇÕES: EMBEDDING

- Nesta abordagem, você pode armazenar o documento relacionado diretamente dentro do documento principal. Isso é útil quando os dados relacionados são acessados frequentemente juntos e você deseja evitar consultas separadas.
- **Exemplo:**
 - Aqui, em vez de armazenar o **ObjectId** do usuário em **orders**, você pode incorporar as informações do usuário diretamente no pedido.

```
29 {
30   "_id" : ObjectId("64ac9f9c75e7a9c7a6765715"),
31   "item" : "Laptop",
32   "price" : 1200,
33   "user" : {
34     "name" : "Alice"
35   }
36 }
```

VISÃO GERAL SOBRE AGREGAÇÕES

Como funciona a “chave estrangeira”

OPERADOR PARA CHAVE ESTRANGEIRA

- No MongoDB, a pipeline de agregação é uma poderosa ferramenta que permite processar e transformar documentos em várias etapas.
- O operador **\$lookup** é uma das ferramentas principais para relacionar duas coleções, mas existem muitos outros operadores que permitem realizar transformações complexas.

COMO JUNTAR DUAS COLEÇÕES: REFERENCE

- Nesta abordagem, em vez de armazenar todos os dados relacionados em um único documento, você armazena o `ObjectId` de um documento em outra coleção. Isso é similar a uma chave estrangeira em bancos de dados relacionais.

```
{} aula7.json 2 x
{} aula7.json > ...
1  {
2    "_id": ObjectId("64ac9f0375e7a9c7a6765710"),
3    "name": "Alice"
4  }
5
6  [
7    {
8      "_id": ObjectId("64ac9f9c75e7a9c7a6765715"),
9      "item": "Laptop",
10     "price": 1200,
11     "user_id": ObjectId("64ac9f0375e7a9c7a6765710") // Referência ao usuário Alice
12   }
13 ]
```

Coleção Users

Coleção Orders

COMO JUNTAR DUAS COLEÇÕES: REFERENCE

```
13 db.orders.findOne({_id: ObjectId("64ac9f9c75e7a9c7a6765715")})
14 db.users.findOne({_id: ObjectId("64ac9f0375e7a9c7a6765710")})
15
16 db.orders.aggregate([
17     {
18         $lookup: {
19             from: "users",
20             localField: "user_id",
21             foreignField: "_id",
22             as: "user_info"
23         }
24     }
25 ])
```

COMO JUNTAR DUAS COLEÇÕES: EMBED OU REFERENCE

- **COMPARAÇÃO:**

- Referências são úteis quando você tem uma estrutura de dados mais complexa e deseja normalizar os dados para evitar duplicações.
- Incorporação é ideal para dados que não mudam frequentemente e que são acessados juntos, o que melhora o desempenho de leitura.

- **QUAL ABORDAGEM ESCOLHER?**

- Depende do caso de uso. Se você frequentemente consulta dados de uma entidade em conjunto com os dados relacionados, a incorporação pode ser melhor. Se os dados mudam frequentemente ou são acessados separadamente, o uso de referências pode ser preferível.

USANDO OPERADOR \$GROUP

```
38 db.orders.aggregate([
39   {
40     $group: {
41       _id: "$product_id", // Agrupa pelos IDs de produtos
42       total_orders: { $sum: 1 }, // Conta o número de pedidos
43       total_quantity: { $sum: "$quantity" } // Soma a quantidade de cada pedido
44     }
45   }
46 ])
```

APROFUNDANDO EM AGREGAÇÕES

Mais do que juntar coleções

VISÃO GERAL DAS AGREGAÇÕES NO MONGODB

- A agregação no MongoDB é um **processo que transforma e combina dados de múltiplos documentos para produzir resultados agregados**, como **totais, médias, contagens** e muito mais.
- Diferente das consultas simples (**find**), as agregações permitem realizar operações complexas que envolvem múltiplas etapas de processamento.

VISÃO GERAL DAS AGREGAÇÕES NO MONGODB

- A framework de agregação do MongoDB é composta por uma série de estágios que processam os documentos sequencialmente.
- **Cada estágio realiza uma operação específica e passa o resultado para o próximo estágio na pipeline.**
- Essa abordagem modular oferece grande flexibilidade e poder na manipulação de dados.

POR QUE USAR AGREGAÇÕES?

- **Análise de Dados:**
 - Facilita a realização de análises complexas diretamente no banco de dados.
- **Desempenho:**
 - Reduz a necessidade de transferir grandes volumes de dados para a aplicação para processamento.
- **Flexibilidade:**
 - Permite transformar e combinar dados de maneiras diversas.
- **Funcionalidades Avançadas:**
 - Suporta operações como joins (**\$lookup**), agrupamentos (**\$group**), ordenações e muito mais.

PIPELINE DE AGREGAÇÃO

- **A pipeline de agregação é a sequência de estágios que os documentos percorrem durante o processo de agregação. Cada estágio aplica uma operação específica aos documentos e passa o resultado para o próximo estágio.**

```
1  db.collection.aggregate([
2      { estágio1 },
3      { estágio2 },
4      { estágio3 },
5      // ...
6  ])
```

FLUXO DE DADOS NA PIPELINE

- **Entrada:**
 - A pipeline recebe os documentos da coleção especificada.
- **Processamento:**
 - Cada estágio processa os documentos conforme a sua função.
- **Saída:**
 - O resultado final é retornado após todos os estágios terem sido aplicados.

EXEMPLO DE FLUXO DE DADOS NA PIPELINE

```
1 db.vendas.aggregate([
2   { $match: { ano: 2023 } }, // Filtra documentos
3   { $group: { _id: "$mes", total: { $sum: "$valor" } } }, // Agrupa e soma
4   { $sort: { total: -1 } } // Ordena os resultados
5 ])
```

ESTÁGIOS DE AGREGAÇÃO

- Os estágios de agregação são as operações que compõem a pipeline. Cada estágio realiza uma transformação nos documentos e passa o resultado para o próximo estágio.
- O estágio **\$match** filtra os documentos de acordo com os critérios especificados, **semelhante à cláusula WHERE em SQL**:

```
1 db.vendas.aggregate([  
2   { $match: { ano: 2023 } }  
3 ])
```

ESTÁGIOS DE AGREGAÇÃO

- O estágio **\$group** agrupa documentos com base em um ou mais campos e permite calcular agregados como soma, média, contagem, etc.

```
1 db.vendas.aggregate([
2   { $group: { _id: "$mes", total_vendas: { $sum: "$valor" } } }
3 ])
```

ESTÁGIOS DE AGREGAÇÃO

- O estágio **\$project** permite selecionar, incluir ou excluir campos específicos nos documentos resultantes. Também pode ser usado para criar novos campos ou transformar dados.

```
1 db.vendas.aggregate([
2   { $project: { nome: 1, valor: 1, _id: 0 } }
3 ])
```

ESTÁGIOS DE AGREGAÇÃO

- O estágio **\$sort** ordena os documentos com base em um ou mais campos.

```
1 db.vendas.aggregate([
2   { $sort: { valor: -1 } } // -1 forma decrescente / 1 forma crescente
3 ])
```

ESTÁGIOS DE AGREGAÇÃO

- **\$limit**: Restringe o número de documentos que passam para os estágios seguintes.
- **\$skip**: Ignora um número especificado de documentos.

```
1 db.vendas.aggregate([
2   { $sort: { valor: -1 } },
3   { $limit: 5 }
4 ])
```

ESTÁGIOS DE AGREGAÇÃO

- O estágio `$unwind` desestrutura um array, criando um documento para cada elemento do array.

```
1 db.pedidos.aggregate([  
2   { $unwind: "$itens" }  
3 ])
```

ESTÁGIOS DE AGREGAÇÃO

- O estágio **\$facet** permite executar múltiplas pipelines de agregação em paralelo e combinar os resultados.

```
1 db.vendas.aggregate([
2   {
3     $facet: {
4       total_vendas: [{ $count: "count" }],
5       soma_total: [{ $group: { _id: null, total: { $sum: "$valor" } } }]
6     }
7   }
8 ])
```

ESTÁGIOS DE AGREGAÇÃO

- **\$bucket**: Agrupa documentos em intervalos predefinidos.
- **\$bucketAuto**: Agrupa documentos em um número especificado de buckets automaticamente.

```
1 db.vendas.aggregate([
2   {
3     $bucket: {
4       groupBy: "$valor",
5       boundaries: [0, 100, 200, 300],
6       default: "Mais de 300",
7       output: { total_vendas: { $sum: 1 }, soma_valores: { $sum: "$valor" } }
8     }
9   }
10  ])
```

ESTÁGIOS DE AGREGAÇÃO

- **\$addFields**: Adiciona novos campos aos documentos.
- **\$set**: Similar ao **\$addFields**, mas também pode modificar campos existentes.

```
1 db.vendas.aggregate([
2   {
3     $addFields: {
4       total: { $multiply: ["$quantidade", "$preco_unitario"] }
5     }
6   }
7 ])
```

ESTÁGIOS DE AGREGAÇÃO

- O estágio **\$count** adiciona um campo com o número total de documentos que passaram pelo estágio anterior.

```
1 db.vendas.aggregate([  
2   { $count: "total_vendas" }  
3 ])
```

COMO TESTAR?

```
1 db.vendas.aggregate([
2   { $match: { ano: 2023 } },
3   { $group: { _id: "$mes", total_vendas: { $sum: "$valor" } } },
4   { $sort: { total_vendas: -1 } }
5 ]).explain("executionStats")
```

OPERADORES DE AGREGAÇÃO

- **\$sum**: Soma os valores.
- **\$avg**: Calcula a média.
- **\$min**: Encontra o valor mínimo.
- **\$max**: Encontra o valor máximo.
- **\$first**: Retorna o primeiro valor.
- **\$last**: Retorna o último valor.

OPERADORES DE AGREGAÇÃO

- Média de vendas do mês

```
1 db.vendas.aggregate([
2   { $group: { _id: "$mes", media_vendas: { $avg: "$valor" } } }
3 ])
```

OPERADORES CONDICIONAIS

- Permitem realizar operações condicionais, semelhantes a estruturas if-else:
 - **\$cond**: Estrutura condicional.
 - **\$ifNull**: Retorna um valor se o campo for nulo ou indefinido.
 - **\$switch**: Implementa uma série de condições.

OPERADORES CONDICIONAIS

```
1 db.vendas.aggregate([
2   {
3     $addFields: {
4       acima_da_media: {
5         $cond: { if: { $gt: ["$valor", 1000] }, then: true, else: false }
6       }
7     }
8   }
9 ])
```

OPERADORES DE ARRAY

- **\$push**: Adiciona elementos a um array.
- **\$addToSet**: Adiciona elementos únicos a um array.
- **\$filter**: Filtra elementos de um array.
- **\$map**: Aplica uma expressão a cada elemento de um array.
- **\$reduce**: Reduz um array a um único valor.

OPERADORES DE ARRAY

```
1 db.pedidos.aggregate([
2   {
3     $project: {
4       itens_filtrados: {
5         $filter: {
6           input: "$itens",
7           as: "item",
8           cond: { $gt: ["$$item.quantidade", 2] }
9         }
10      }
11    }
12  }
13 ])
```

EXERCÍCIOS

```
1 db.cliente.insertMany([{
2     "_id": 153,
3     "nome": "Alice",
4     "email": "alice@example.com"
5 }]);
6
7 db.venda.insertMany([{
8     "_id": 57,
9     "cliente_id": 153,
10    "data_venda": ISODate("2023-01-15T08:00:00Z"),
11    "mes": 1,
12    "ano": 2023
13 }]);
14
15 db.item.insertMany([{
16     "_id": 1,
17     "venda_id": 57,
18     "produto": "Laptop",
19     "quantidade": 2,
20     "preco_unitario": 1200
21 }]);
```

EXERCÍCIOS

1) Contagem de Vendas por Cliente:

- **Objetivo:** Calcular quantas vendas cada cliente realizou.
- **Dica:** Use \$group com cliente_id.

2) Média de Vendas por Produto:

- **Objetivo:** Determinar a média de vendas para cada tipo de produto.
- **Dica:** Agrupe por produto e utilize \$avg.

EXERCÍCIOS

- **Listar Clientes que Compraram Mais de 5 Produtos:**
 - **Objetivo:** Identificar clientes que realizaram grandes pedidos.
 - **Dica:** Use \$match após \$group.
- **Top 3 Produtos Mais Vendidos:**
 - **Objetivo:** Encontrar os produtos com maior número de vendas.
 - **Dica:** Agrupe por produto, some quantidade e use \$sort seguido de \$limit.
- **Total de Vendas por Região:**
 - **Objetivo:** Se houver um campo regioao em clientes, calcular o total de vendas por região.
 - **Dica:** Utilize \$lookup para unir pedidos e clientes, depois agrupe por regioao.

OBRIGADO!

- Encontre este **material on-line** em:
 - Slides: Google Class Room
- Em caso de **dúvidas**, entre em contato:
 - **Prof. Tiago:** tiago.silva@fatecjahu.edu.br

